

Chapter 10

Quantization Techniques

Exam Study Guide - Chapter 2 Style Flow

Based on the Chapter 10 lecture and the Chapter 10 Summary and Sample Questions.

Item	What this guide gives you
Focus	Quantization as numerical approximation, not only compression.
Exam style	Definitions, symbolic equations, conceptual tradeoffs, method comparison, and failure-mode reasoning.
Main flow	Motivation -> core quantizer -> error sources -> three phases -> QAT/PTQ methods -> generative and multimodal quantization.

How to Study This Chapter

Read Chapter 10 as one coherent story: low precision gives efficiency, but it creates approximation error. Every method in the chapter is a different answer to where, when, and how that error should be handled.

Section	Purpose
1. Quick exam map	The highest-priority topics and what they test.
2. Summary-to-lecture matching map	Where each summary idea appears in the full lecture flow.
3. Concept sections	Clean explanation of the main ideas with symbolic equations.
4. Method comparison	QAT vs PTQ and GPTQ/SmoothQuant/AWQ/OmniQuant/I-BERT/QLoRA.
5. Sample final questions	Polished answers to the summary questions.
6. Flashcards	Fast revision after redundant wording is removed.
7. Last-minute checklist	One-page final review.

1. Quick Exam Map

Priority	Topic	What you must explain
Very high	Uniform affine quantization	q , $\hat{x}$, s , z , $q_{\min}$, $q_{\max}$, b , clipping, and reconstruction.
Very high	Scale-range tradeoff	Small s improves resolution but clips; large s covers range but increases rounding error.
Very high	Three phases	Pre-quantization, during-training quantization/QAT, and post-training quantization/PTQ.
Very high	STE	Why rounding/clipping block true gradients and why STE supplies a surrogate.
High	PACT and LSQ	PACT learns α ; LSQ learns s . Both make quantizer choices trainable.
High	Hessian sensitivity	Use $\Delta L \approx \frac{1}{2} \mathbf{e}^T \mathbf{H} \mathbf{e}$ to explain why equal-size errors can be unequally harmful.
High	GPTQ	Quantize $\rightarrow$ estimate damage $\rightarrow$ compensate remaining weights with inverse-curvature information.
High	Transformer/LLM PTQ	Compare SmoothQuant, AWQ, OmniQuant, I-BERT, and QLoRA conceptually.
Medium	Generative/multimodal models	Explain timestep, trajectory, KV-cache, and cross-modal alignment issues.

2. Summary-to-Lecture Matching Map

This map links the abstract summary document to the corresponding detailed lecture sections.

Summary idea	Matching lecture section	Exam takeaway
Quantization as approximation	Part I: Problem formulation; Why do quantization?	Efficiency is useful only if task behavior is preserved.
Uniform quantizer	Part II: Uniform affine quantization	Memorize $q = \text{clip}(\text{round}(x/s)+z, q_{\min}, q_{\max})$ and $x_{\text{hat}} = s(q-z)$.
Scale-range tension	Tradeoff between scale and range; extreme examples	The same s controls resolution and real-valued coverage.
What/how to quantize	Per-tensor, per-channel, group-wise; quantized linear layers	Weights, activations, KV cache, embeddings, and operators may need different treatment.
Pre-quantization	Calibration and sensitivity analysis	Measure ranges, outliers, and fragile layers before conversion.
Curvature/Hessian	Second-order Taylor view; Hessian examples	Loss damage depends on direction, not only error magnitude.
QAT	During-training quantization; fake quantization; STE	Train while forward pass experiences quantization distortion.
PACT/LSQ	Learned clipping and step-size sections	Previously fixed numerical choices become trainable.
PTQ	RTN, AdaRound, GPTQ, transformer-specific PTQ	Convert pretrained models with little/no retraining.
Generative/multimodal difficulty	Why Quantization Is Difficult across model families	Error can accumulate over time or disturb multimodal alignment.

3. Motivation and Formulation

Modern models are usually trained in FP32, BF16, or FP16, but deployment is limited by memory footprint, memory bandwidth, and arithmetic throughput. Quantization reduces the number of bits used to represent values, ideally making models smaller, faster, and cheaper to run.

```

M_l = n_l b_l bits
M_total = Σ_{l=1}^L n_l b_l
FP32 -> INT8 ideal parameter compression = 32 / 8 = 4x
FP32 -> INT4 ideal parameter compression = 32 / 4 = 8x

```

Quantization is not only storage compression. Real deployment gains require hardware-compatible low-precision kernels, controlled dequantization overhead, and careful treatment of activation outliers.

Constrained approximation view

```

Q(W) = argmin_{W_hat in Q_set} D(W, W_hat)
D(W, W_hat) = ||W - W_hat||_2^2
Delta L ≈ L(W_hat) - L(W)

```

Simple Euclidean weight error is often too naive. A small perturbation in a sensitive direction can damage the loss more than a larger perturbation in a flat direction.

Second-order loss view

```

L(W_hat) ≈ L(W) + grad L(W)^T (W_hat - W) + 1/2 (W_hat - W)^T H (W_hat - W)
Near a trained optimum: grad L(W) ≈ 0
Delta L ≈ 1/2 e^T H e, where e = W_hat - W

```

This equation is the conceptual bridge from naive rounding to second-order PTQ methods such as GPTQ.

4. Core Quantizer and Symbols

Uniform affine quantization

```
q = clip(round(x / s) + z, q_min, q_max)
x_hat = s(q - z)
```

Symbol	Meaning
x	Original real-valued value/tensor.
x_hat	Dequantized reconstruction.
q	Stored integer code.
s	Scale / step size.
z	Zero-point.
b	Bit-width.
q_min, q_max	Integer code limits.
α	Clipping threshold.
Q	Largest positive level in symmetric quantization.

Symmetric signed quantization

```
z = 0
Q = 2^(b-1) - 1
q = clip(round(x / s), -Q, Q)
x_hat = s q
s = max|x| / Q
```

Symmetric quantization is simple and useful when values are centered around zero, but outliers can make s large and ordinary values poorly represented.

Scale-range tradeoff

```
Representable levels = {s q_min, s(q_min + 1), ..., s q_max}
Grid spacing = s
Representable interval = [s q_min, s q_max]
```

Scale choice	Benefit	Failure mode
s too small	Fine grid and low rounding error for small values.	Range is narrow, so many values clip.
s too large	Wide range and less clipping.	Grid is coarse, so rounding error grows.
balanced s	Acceptable clipping and rounding.	Usually needs calibration or learned/optimized scales.

Error decomposition

```
e = x - x_hat
MSE = E[(x - x_hat)^2]
x_clip = clip(x, -alpha, alpha)
MSE_total ≈ MSE_round + MSE_clip
```

Rounding error comes from the discrete grid. Clipping error comes from values outside the representable interval.

5. Granularity and Quantized Linear Layers

Scheme	Formula idea	Strength	Weakness
Per-tensor	$W_{\text{hat}} = s * \text{round}(W/s)$	Simple and hardware-friendly.	Bad when channels have very different ranges.
Per-channel	$W_{\text{hat}}[i,:] = s_i * \text{round}(W[i,:]/s_i)$	Captures channel-specific ranges.	More scale metadata.
Group-wise	$W_{\text{hat}}[G_k] = s_k * \text{round}(W[G_k]/s_k)$	Good transformer compromise.	Requires group-size design.

$$\begin{aligned}
 y &= W x + b \\
 W &\approx s_W Q_W, \quad x \approx s_x Q_x \\
 y &\approx s_W s_x (Q_W Q_x) + b
 \end{aligned}$$

The expensive operation becomes integer matrix multiplication $Q_W Q_x$, followed by scaling. Practical benefits depend on backend support.

6. The Three Phases of Quantization

Phase	When	Main question	Tools
Pre-quantization	Before low precision is imposed.	Which layers are sensitive and what design should we choose?	Calibration, observers, outlier analysis, sensitivity, mixed precision.
QAT / during-training	During training or fine-tuning.	Can the model learn to tolerate quantization noise?	Fake quantization, STE, PACT, LSQ, binary/ternary training.
PTQ / post-training	After full-precision training.	Can we convert cheaply without full retraining?	RTN, AdaRound, GPTQ, SmoothQuant, AWQ, OmniQuant, I-BERT.

6.1 Pre-quantization

Pre-quantization is preparation. It studies activation ranges, outliers, layer sensitivity, calibration data, and granularity before the model is converted.

- Choose representative calibration data.
- Estimate min-max, percentiles, or channel-wise ranges.
- Decide what to quantize: weights, activations, KV cache, embeddings, or selected layers.
- Choose per-tensor, per-channel, group-wise, and mixed precision.
- Keep fragile layers or operators in higher precision.

```
s_minmax = max_{a in C} |a| / Q
s_percentile = Percentile_p(|a|) / Q
```

6.2 Sensitivity and Hessian reasoning

$$\Delta L_l \approx 1/2 \text{vec}(\Delta W_l)^T H_l \text{vec}(\Delta W_l)$$

A layer is sensitive when its perturbation is large or when the curvature H_l is large in the perturbation direction.

6.3 QAT: training under quantized behavior

$$\min_{\{W_l, \theta_l^W, \theta_l^a\}} L(f(x; \{Q_W(W_l; \theta_l^W)\}, \{Q_a(a_l; \theta_l^a)\}))$$

QAT simulates quantization during the forward pass so the model learns weights that still work under low precision.

6.4 STE

Forward: $x_q = \text{fake_quant}(x)$

Backward: $d x_q / d x \approx 1$ inside clipping range, often 0 outside

STE is not the true derivative. It is a practical surrogate that lets gradients pass through a discrete operation.

6.5 PACT, LSQ, binary, ternary

Method	What changes	Core idea
PACT	Learns clipping threshold α .	The clipping range becomes a trainable decision.
LSQ	Learns step size s .	The grid spacing becomes a trainable decision.
Binary	Uses mostly sign information.	Extreme compression; very small alphabet.
Ternary	Uses $\{-1, 0, +1\}$.	Adds zero state and sparsity.

6.6 PTQ: converting a trained model

Method	Main idea	What it fixes
RTN	Round to nearest level.	Simple baseline, but ignores data and sensitivity.
AdaRound	Data-driven rounding with layer reconstruction.	Preserves layer output better than scalar rounding.
GPTQ	Second-order curvature + compensation.	Adjusts remaining weights after quantizing a block.
SmoothQuant	Equivalent scaling $y = Wx = (W S^{-1})(Sx)$.	Moves activation outlier difficulty into weights.
AWQ	Activation-aware weight protection.	Protects salient channels.
OmniQuant	Learnable clipping and equivalent transformations.	Makes calibration adaptive inside PTQ.
I-BERT	Integer-friendly GELU/Softmax/LayerNorm.	Moves toward a fully integer transformer graph.

7. Important Methods in Human Words

GPTQ

GPTQ is second-order PTQ. It does not simply round each block and accept the error. It quantizes a block, estimates the damage using curvature, and adjusts remaining weights so part of the damage is absorbed.

Core idea: quantize -> estimate damage -> compensate -> continue
Damage model: $\Delta L \approx \frac{1}{2} e^T H e$

SmoothQuant

$$y = Wx = (WS^{-1})(Sx)$$

The function is unchanged in floating point, but the numerical difficulty is redistributed. Activation outliers become easier to quantize because part of the difficulty is moved into weights.

AWQ

AWQ uses activations to find which weight channels matter most during real inference. These salient channels are protected so low-bit quantization damages the model less.

OmniQuant

OmniQuant is PTQ with learnable calibration ingredients. It learns clipping and equivalent transformations without full end-to-end retraining, so it is more adaptive than fixed-rule PTQ.

I-BERT

I-BERT matters because quantizing only matrix multiplications is not enough. A deployable integer model also needs integer-friendly GELU, Softmax, and LayerNorm.

QLoRA

QLoRA keeps a low-bit frozen base model and trains small adapters. It is a practical path for low-memory fine-tuning rather than only a final deployment trick.

8. Generative and Multimodal Quantization

Model family	Why it is hard	Good first response
LLMs	Activation outliers, residual streams, attention, normalization, and KV cache.	Use group-wise/outlier-aware PTQ, GPTQ, AWQ, SmoothQuant, OmniQuant, KV-cache analysis.
Diffusion models	Denoiser errors accumulate over many timesteps; statistics shift with timestep.	Use timestep-aware calibration and trajectory-quality metrics.
DiT	Transformer outliers plus diffusion timestep dependence.	Use block/timestep sensitivity and mixed precision.
Flow matching	Low-bit errors perturb a time-dependent velocity field and ODE path.	Use time-state and solver-aware calibration.
CLIP-like models	Need to preserve image-text embedding geometry.	Evaluate retrieval and zero-shot alignment, not only local accuracy.
Fusion VLMs	Vision encoder, projector, fusion blocks, and decoder have unequal sensitivities.	Use modality-aware and token-aware PTQ.

Algorithmic template

- 1 Identify the repeated update: token step, denoising step, or ODE step.
- 2 Build calibration data that matches deployment states.
- 3 Measure layer, token, timestep, and modality sensitivity.
- 4 Assign mixed precision when needed.
- 5 Choose the method: RTN, GPTQ, AWQ, SmoothQuant, OmniQuant, timestep-aware PTQ, or solver-aware PTQ.
- 6 Evaluate iterative degradation, not only static single-step error.

9. Equations Cheat Sheet

Storage:

$M_l = n_l \cdot b_l$ bits

$M_{\text{total}} = \sum_{l=1}^L n_l \cdot b_l$

Affine quantization:

$q = \text{clip}(\text{round}(x / s) + z, q_{\min}, q_{\max})$

$x_{\text{hat}} = s(q - z)$

Symmetric signed quantization:

$Q = 2^{(b-1)} - 1$

$q = \text{clip}(\text{round}(x / s), -Q, Q)$

$x_{\text{hat}} = s \cdot q$

$s = \max|x| / Q$

Scale and range:

levels = $\{s \cdot q_{\min}, s(q_{\min} + 1), \dots, s \cdot q_{\max}\}$

spacing = s

range = $[s \cdot q_{\min}, s \cdot q_{\max}]$

Error:

$e = x - x_{\text{hat}}$

$\text{MSE} = E[(x - x_{\text{hat}})^2]$

$\text{MSE}_{\text{total}} \approx \text{MSE}_{\text{round}} + \text{MSE}_{\text{clip}}$

Second order:

$L(W_{\text{hat}}) \approx L(W) + \text{grad } L(W)^T (W_{\text{hat}} - W) + 1/2 (W_{\text{hat}} - W)^T H(W_{\text{hat}} - W)$

If $\text{grad } L(W) \approx 0$: $\Delta L \approx 1/2 e^T H e$

QAT objective:

$\min_{\{W_l, \theta_l^W, \theta_l^a\}} L(f(x; \{Q(W_l; \theta_l^W)\}, \{Q(a_l; \theta_l^a)\}))$

SmoothQuant:

$y = Wx = (W S^{-1})(Sx)$

10. Sample Final Exam Questions

Question 1

What is the central tradeoff between scale and range?

Sample answer: Scale s controls both the spacing between levels and the total representable real range. Small s gives fine resolution but narrow range and clipping. Large s gives wide range but coarse resolution and more rounding error. The tension is structural because one parameter controls two competing goals.

Question 2

Why is STE necessary for QAT?

Sample answer: The forward quantizer uses rounding and clipping, whose true derivatives are unusable for gradient descent. STE uses quantized behavior in the forward pass but a surrogate derivative in the backward pass so training can continue.

Question 3

Why are PACT and LSQ examples of learning numerical decisions?

Sample answer: PACT learns the clipping threshold α , and LSQ learns the step size s . Both turn fixed quantizer settings into trainable variables optimized for the task.

Question 4

Why does GPTQ compensate after quantizing a block?

Sample answer: Because the remaining weights can still move. Under a quadratic loss model, inverse-curvature information gives a compensating update that reduces the damage caused by the newly quantized block.

Question 5

Why are diffusion models harder to quantize than ordinary classifiers?

Sample answer: Errors in a classifier propagate through one forward pass, but diffusion denoisers are called repeatedly across timesteps. Errors can compound, and activation distributions change with timestep, so static calibration can fail.

11. Exam Flashcards

Question	Answer
1. What is quantization?	Mapping high-precision values to low-precision discrete levels while preserving model behavior.
2. Why quantize?	Memory, bandwidth, latency, energy, and deployability.
3. What is q ?	The stored integer code.
4. What is s ?	The scale or step size.
5. What is z ?	The zero-point.
6. What is $\hat{x}$?	The reconstructed value after dequantization.
7. Affine quantization formula?	$q = \text{clip}(\text{round}(x/s) + z, q_{\min}, q_{\max})$.
8. Reconstruction formula?	$\hat{x} = s(q - z)$.
9. Symmetric quantization?	$z=0$ and signed integer range $[-Q, Q]$.
10. Signed b -bit Q ?	$Q = 2^{(b-1)} - 1$.
11. If s is too small?	Fine resolution but severe clipping.

Question	Answer
12. If s is too large?	Wide range but high rounding error.
13. Two error sources?	Rounding error and clipping error.
14. What is α ?	Clipping threshold.
15. Per-tensor quantization?	One scale for the whole tensor.
16. Per-channel quantization?	One scale per output channel.
17. Group-wise quantization?	One scale per group of channels/columns.
18. What is calibration?	Estimating ranges/statistics from representative data.
19. Why is calibration important?	Bad calibration gives wrong scales at deployment.
20. What is sensitivity analysis?	Finding which layers/components are harmed most by quantization.
21. What is mixed precision?	Different bit-widths for different components.
22. What is the Hessian?	Matrix of second derivatives of the loss.

Question	Answer
23. Why use the Hessian?	It measures curvature and quantization damage directions.
24. Core second-order damage formula?	$\Delta L \approx \frac{1}{2} e^T H e$.
25. What is QAT?	Training while the forward pass includes quantization effects.
26. What is fake quantization?	Simulated quantization during training.
27. Why STE?	To pass useful gradients through rounding/clipping.
28. PACT learns what?	Clipping threshold α .
29. LSQ learns what?	Step size s .
30. What is PTQ?	Post-training conversion to low precision.
31. What is RTN?	Round-to-nearest baseline.
32. AdaRound improves what?	Rounding decisions using calibration data.
33. GPTQ adds what?	Curvature-aware compensation.

Question	Answer
34. SmoothQuant does what?	Rescales activations/weights to handle outliers.
35. AWQ protects what?	Activation-salient weight channels.
36. OmniQuant adds what?	Learnable clipping and equivalent transformations.
37. I-BERT focuses on what?	Integer-only transformer inference, including nonlinear ops.
38. QLoRA combines what?	Low-bit base model and trainable adapters.

Question	Answer
39. Why are LLMs hard?	Outliers, attention, residuals, normalization, KV cache.
40. Why are diffusion models hard?	Timestep-dependent statistics and error accumulation.
41. Why is flow matching hard?	Low-bit errors distort the velocity field and ODE path.
42. Why is CLIP hard?	Must preserve cross-modal embedding geometry.
43. Why are VLMs hard?	Different modalities and fusion blocks have unequal sensitivities.
44. Big lesson?	Quantization chooses how to absorb approximation error while preserving useful behavior.

12. Last-Minute Checklist

Checklist item	Done
I can write and explain $q = \text{clip}(\text{round}(x/s)+z, q_{\min}, q_{\max})$.	☐
I can explain why one scale controls both spacing and range.	☐
I can distinguish rounding error and clipping error.	☐
I can compare per-tensor, per-channel, and group-wise scales.	☐
I can define pre-quantization, QAT, and PTQ.	☐
I can explain STE as a surrogate gradient, not the true derivative.	☐
I can compare PACT and LSQ.	☐
I can explain $\Delta L \approx 1/2 e^T H e$.	☐
I can explain GPTQ as quantize-estimate-compensate.	☐
I can compare SmoothQuant, AWQ, OmniQuant, I-BERT, and QLoRA.	☐
I can explain why diffusion and VLM quantization are harder than simple classifier quantization.	☐

One paragraph to memorize

Quantization maps high-precision weights and activations to discrete low-precision levels. The core formula $q = \text{clip}(\text{round}(x/s)+z, q_{\min}, q_{\max})$ shows the main tension: small s gives fine resolution but narrow range, while large s gives wider range but coarser approximation. Pre-quantization measures ranges, outliers, and sensitivity; QAT trains under fake quantization using STE; PTQ converts pretrained models using calibration, reconstruction, curvature, and equivalent transformations. Modern methods such as GPTQ, SmoothQuant, AWQ, OmniQuant, I-BERT, and QLoRA are different strategies for preserving behavior under low precision, especially in transformers, generative models, and multimodal systems where errors can accumulate or disturb alignment.